

Partie 07 — Vectorisation et simulation

INF-STAR — Programmation statistique avec R

Oskar Laverny

October 5, 2026

Aix-Marseille Université

Objectifs

Vous saurez :

- reconnaître une opération vectorisée ;
- appliquer une fonction à plusieurs objets avec `lapply()` et `sapply()` ;
- reconnaître l'usage de `apply()` sur une matrice ;
- simuler des données reproductibles avec `set.seed()`.

Vectorisation



R travaille naturellement sur des vecteurs

```
x <- c(2, 4, 6, 8)
x + 1
x^2
sqrt(x)
```

Ces opérations agissent sur tous les éléments sans boucle explicite.

Indexation logique

```
x <- c(-2, 4, -1, 7)
x[x > 0]
x[x < 0] <- 0
x
```

La sélection logique est l'une des formes les plus utiles de vectorisation en R.

Pour une matrice :

```
rowSums(m)  
colSums(m)  
rowMeans(m)  
colMeans(m)
```

Lorsqu'une fonction spécialisée existe, elle est souvent plus lisible et plus rapide qu'une solution générique.

La famille *apply

Pourquoi des fonctionnelles ?

Une fonctionnelle reçoit une fonction en argument et l'applique selon une structure définie.

Exemple conceptuel :

```
plusieurs objets -> même fonction -> plusieurs résultats
```

C'est une autre façon d'exprimer une répétition.

Deux outils à pratiquer

- `lapply()` : une liste de résultats, un par objet.
- `sapply()` : une simplification en vecteur lorsque les résultats le permettent.

`apply()` sera montré sur une matrice. Les autres variantes ne sont pas exigées dans ce parcours ni dans le DM2.

`apply()`



Lignes ou colonnes d'une matrice

```
m <- matrix(1:12, nrow = 3)

apply(m, MARGIN = 1, FUN = mean)
apply(m, MARGIN = 2, FUN = mean)
```

- MARGIN = 1 : lignes ;
- MARGIN = 2 : colonnes.

Arguments supplémentaires

```
m <- matrix(c(1, NA, 3, 4, 5, 6), nrow = 2)

apply(m, 2, mean, na.rm = TRUE)
```

Les arguments après FUN sont transmis à la fonction appliquée.

`lapply()`

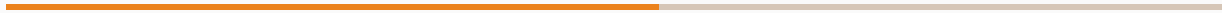


Toujours une liste en sortie

```
x <- list(  
  a = 1:5,  
  b = c(10, 20, 30),  
  c = c(2, 4)  
)  
  
lapply(x, mean)
```

La structure de sortie est prévisible : une liste.

`sapply()`



Simplifier quand c'est possible

```
sapply(x, mean)
```

Si chaque résultat est un scalaire compatible, `sapply()` retourne souvent un vecteur.

La simplification dépend des résultats : vérifiez toujours la forme obtenue.

Plusieurs colonnes d'un tableau

```
donnees <- data.frame(age = c(35, 50, 65), tas = c(120, NA, 145))  
sapply(donnees, mean, na.rm = TRUE)  
lapply(donnees, summary)
```

Un tableau est aussi une liste de colonnes. Choisir les colonnes numériques avant de calculer une moyenne.

Réutiliser une fonction personnelle

```
etendue <- function(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
sapply(donnees, etendue)
sapply(donnees, function(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
```

Une fonction anonyme convient à une opération courte utilisée une seule fois.

Quelle écriture choisir ?

- Carrés d'un vecteur : x^2 .
- Résumé de chaque colonne : `lapply()`.
- Moyenne de chaque colonne : `sapply()`.
- Moyennes des colonnes d'une matrice : `colMeans()`.
- Calcul dépendant de l'étape précédente : boucle.

Simulation



Aléatoire, mais reproductible

```
set.seed(123)  
rnorm(5)
```

`set.seed()` fixe l'état du générateur pseudo-aléatoire : un autre étudiant peut alors reproduire la même suite de tirages dans le même environnement. Fixez la graine avant l'expérience, pas à chaque répétition.

Quelques générateurs

```
rnorm(100, mean = 120, sd = 15)
runif(100, min = 0, max = 1)
rbinom(100, size = 1, prob = 0.3)
sample(c("A", "B"), size = 20, replace = TRUE)
```

Choisissez une loi qui correspond au mécanisme que vous souhaitez représenter.

Répéter une simulation

```
set.seed(123)

replicate(
  1000,
  mean(rnorm(50, mean = 120, sd = 15))
)
```

`replicate()` est très pratique pour répéter une expression et rassembler ses résultats.

Effet de la taille d'échantillon

```
set.seed(123)

m10 <- replicate(1000, mean(rnorm(10)))
m100 <- replicate(1000, mean(rnorm(100)))

sd(m10)
sd(m100)
```

Les moyennes d'échantillons plus grands varient moins d'une répétition à l'autre.

Simulation Monte-Carlo simple

Approximer $P(Z > 1.96)$ pour $Z \sim N(0, 1)$:

```
set.seed(123)
z <- rnorm(100000)
mean(z > 1.96)
```

La simulation permet d'estimer une probabilité en répétant une expérience aléatoire.

```
set.seed(123)
moyennes <- numeric(1000)
for (i in seq_along(moyennes)) {
  moyennes[i] <- mean(rnorm(30, mean = 100, sd = 15))
}
```

Chaque résultat est une moyenne d'échantillon, pas une observation individuelle. Reprenez l'expérience avec 300 observations : quel effet sur la dispersion ?

À retenir



Trois manières de répéter

1. **vectoriser** lorsque l'opération existe naturellement sur tout le vecteur ;
2. écrire une **boucle** lorsque le contrôle pas à pas est le plus clair ;
3. utiliser **`lapply()`** **ou** **`sapply()`** pour appliquer la même fonction à une collection structurée.

Aucune n'est « toujours meilleure » : choisissez selon le problème et la lisibilité.